

AWS 與生成式 AI 課程 · 單元 D01

學習方法與 AI 家教

盧建霖 • 2026.09.17

12

個月，就有一半的答案要重查

與其背今天的答案，先練會判斷、查證、記住與除錯的流程。這九課就是這套流程。

Agenda

- 01 判斷資訊：來源、文件、該不該學
- 02 跟 AI 一起學：陷阱與家教
- 03 留得住：記憶與除錯
- 04 做得動：時間與環境

D01

學習方法與 AI 家教

九課 · 460 分鐘

每課一份檢查點與個人作業

LESSON 1

資訊來源分三層

眼前這篇文章，到底能不能信？

一、 一手、二手、三手，各拿來做什麼

二、 過期教學的六個訊號

三、 判讀順序：內文最後看

離源頭越遠，越可能過期

每轉一手，可信度與更新速度各掉一級

PRIMARY

一手

做這個東西的人自己寫的：官方文件、
CHANGELOG、原始碼、證照考綱

跟服務同步更新



SECONDARY

二手

整理一手的人寫的：技術部落格、書、課程影片，包括這門課的教材

從出版日開始倒數



TERTIARY

三手

轉述二手的人寫的：懶人包、內容農場轉載，
還有沒附出處的 **AI 回答**

當場就可能錯

三層的用法不一樣

一手下決定，二手找路，三手只找關鍵字

層	例子	拿來做什麼	保存期限
一手	官方文件、What's New、GitHub Release、證照考綱	查參數、查行為、 下決定	跟服務同步
二手	技術部落格、書、課程影片、本課教材	建地圖、找切入點、看別人踩的雷	約一年；超過 18 個月當過期訊號
三手	轉載站、懶人包、沒附出處的 AI 回答	只拿來找關鍵字，再 回一手查	當場就可能錯

過期教學的六個訊號：中兩個就先關掉

前三個看時間，後三個看現場

01

沒日期，或超過 **18** 個月

看頁首頁尾、網址裡的年月

02

模組路徑對不上

把 `import` 那幾行拿去跟官方文件比

03

用到已改名的 **API**

函式名去官方 Reference 搜一次

04

截圖跟現在的介面不符

開主控台對一眼；按鈕搬家，功能可能也搬了

05

留言說跑不動，作者沒回

拉到最底看留言區，別人替你踩過雷

06

多站一字不差的轉載

抓一句丟搜尋；沒人負責更新的副本

六個全沒中，只代表你還沒發現它過期。訊號能淘汰，不能認證。

判讀順序：內文最後看

前三步 30 秒做完，過了才值得讀



1 日期

頁首、網址、更新時間



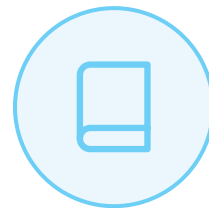
2 路徑

import 與 API 名稱對文件



3 留言

拉到最底，看有沒有人卡住



4 內文

前三關過了才細讀

問 AI 也一樣三件事：[出處在哪](#)、[知識截止到什麼時候](#)、[講的是哪個版本](#)。

LESSON 2

官方文件有三本

我的問題該翻哪一本？怎麼提前知道我學的東西要死了？

- 一、問題型態決定翻哪一本
- 二、讀一頁文件的順序
- 三、只有官方會提前告訴你它要死了

三種問題，翻**三本**不同的文件

上面是你會問的問題，下面是該翻的那一本

我想做到 **X**，
步驟是什麼？



HOW TO

怎麼用

AWS: User Guide、Developer Guide

GCP: Guides、Quickstart

這個參數能填什麼、
預設多少？



REFERENCE

參數是什麼

AWS: API Reference、CLI

Reference、boto3

GCP: REST Reference、gcloud

為什麼這樣設計、
取捨是什麼？



WHY

為什麼

Well-Architected Framework、FAQ、白皮書、各服務 Concepts

Well-Architected Framework: 雲端業者整理的架構設計原則，AWS 版分六個面向。

讀一頁文件：先看框框，先跑例子

文件不是小說，不用從第一行讀

01

版本與日期

網址裡的 **latest**、頁尾的更新時間

02

提示框

Note、**Important**、**Warning**
先讀

03

例子

先照官方例子跑一次，讓它先動

04

參數表

跑通了才逐欄讀

順便改掉三個習慣

搜服務名點第一篇

改成文件站內搜尋，或加 **site:** 限定官方網域

在使用指南翻預設值

改成去 **Reference** 那一本

問 **AI** 怎麼設定然後照抄

改成問它在哪一本、哪一章，連結點進去自己讀

只有官方會提前告訴你它要死了

CodeCommit 是 AWS 自己的 Git 代管服務，Cloud9 是它的雲端程式編輯器



二手作者自己也不一定知道服務要被關，所以每個月掃一次官方的 [What's New](#)。

一手文件的另外兩種讀法

EXAM GUIDE

考綱：從能力敘述反推該讀什麼

SAA-C03 四個 Domain 與配分：安全架構 30%、韌性架構 26%、高效能架構 24%、成本最佳化 20%

Task Statement 1.1：設計對 **AWS** 資源的安全存取

它列的 Knowledge of：跨帳號存取控制、IAM 與 Identity Center、全球基礎架構、最小權限、共同責任模型

每一條 Knowledge 對到使用指南的一章，列出來就是讀書計畫；課堂沒教的那幾章就是你的自習清單。

CHANGELOG

先看會壞的，再看被拿掉的

- | | | |
|---|-------------------------|-------------|
| 1 | Breaking changes | 升級後會壞的 |
| 2 | Deprecated | 預告要拿掉，開始找替代 |
| 3 | Removed | 通常就是你跑不動的原因 |
| 4 | Added | 最後看，順便學新功能 |

版本號 x.y.z：x 變了就一定要讀遷移指南；y 與 z 通常向下相容，Deprecated 那段還是掃一眼。

LESSON 3

技術該不該學的五個問題

新工具每週都有，我怎麼在十分鐘內決定學不學？

- 一、 五個問題，各有一個地方可查
- 二、 概念、通用工具、廠商包裝
- 三、 框架會判錯的三種情況

五個問題，每一題都有可查的地方

不是憑感覺，每題都指到一個網頁

問題	去哪查	紅燈訊號
誰在維護	GitHub 最後一次 commit 、維護者是公司還是一個人	半年沒 commit ；只有一個人在回 issue
有沒有生產採用	官方客戶案例、徵才條件、 Stack Overflow 題數趨勢	只有 demo 與部落格，找不到公司在用
它取代了什麼	官方的「與 X 比較」頁、遷移指南	說不出取代了什麼，或只是換個名字
生命週期怎麼公告	有沒有棄用政策、 What's New 、 release notes	沒有公告機制，功能說消失就消失
學會之後帶得走嗎	概念是不是跨工具、跨雲通用	只在這一家有效的語法與名詞

門檻兩條：一題答不出來，先查到有答案再決定；兩題以上答不出來，等半年再看。

先分清楚：概念、通用工具、廠商包裝

箭頭是學習順序；越往右，越容易被關掉



先概念，再通用工具，包裝版之後一天就上手。star 數是歷史，commit 才是現在。

框架會判錯的三種情況

五問不是萬用，這三種情況要換做法

TOO NEW

太新，五題都答不出

不代表不好，代表還看不出來。記下日期，六個月後重問，這段時間用通用工具頂著。

ALREADY IN USE

公司已經在用

那就學，五問只拿來估風險。把「取代了什麼」問清楚，帶不走的部分少花時間。

ONE WEEKEND

一個週末能試完

成本 8 小時以內可以不問五題，做一個能跑的 **demo** 就知道，試完再決定要不要投 40 小時。

把判斷寫下來：工具、判斷日、五問的答案、判定、六個月後重問。**CodeCommit** 就是現成例子：2024 年 7 月判不學，2025 年 11 月它回歸，重新評估。

QUIZ 1

一篇文章六個過期訊號全沒中，你可以下什麼結論？

A 內容正確，可以照做

B 還沒發現它過期，參數仍要回一手對

C 它是一手來源

D 作者一定是做這個東西的人

想 30 秒，等一下翻下一頁對答案。

QUIZ 1 · 解答

一篇文章六個過期訊號全沒中，你可以下什麼結論？

A 內容正確，可以照做

✓ 還沒發現它過期，參數仍要回一手對

C 它是一手來源

D 作者一定是做這個東西的人

B。訊號只能淘汰，不能認證；A 是最常見的誤用，C、D 把訊號當成在看作者身分。

Agenda

- 01 判斷資訊：來源、文件、該不該學
- 02 跟 AI 一起學：陷阱與家教
- 03 留得住：記憶與除錯
- 04 做得動：時間與環境

02

第 4、5 課

這門課用 AI 的規矩
就在這一段定下來

LESSON 4

用 AI 學習的三個陷阱

它答得那麼順，我為什麼還要去對？

- 一、 三個陷阱，同一個解法
- 二、 現場示範：問它一個新服務
- 三、 全課四步工作流與驗證分類

三個陷阱，同一個解法

中一個就白學；解法都是回一手來源對

知識截止日

模型訓練資料收到哪一天為止。你問它截止之後才出現的服務，它不說不知道，會補一個像的。

新服務、新 **Region**、新價格最常被編

流暢不等於正確

人講錯會卡住、會遲疑，它不會。錯答案跟對答案語氣一樣、格式一樣。

唯一的分辨法是去對一手

跳過理解

程式能跑，但你答不出「這行為什麼這樣寫」。抄來沒改過的每一行都是負債。

面試第一個問這個

解法只有一個：**回第 1 課講的一手來源對**。

現場示範：問它一個**2025**年才有的 **Region**

Region 是一個地理區域；可用區是區域裡互相獨立的機房群；**opt-in** 指新 **Region** 預設關閉，要手動啟用

問它	示範時可能出現的回答	官方文件怎麼說
台北 Region 的代號？	答對 ap-east-2 ，或說「AWS 沒有台北 Region 」	ap-east-2 ， 2025 年開通
有幾個可用區？	編 2 個、3 個、4 個都見過	3 個 Availability Zone
新帳號直接能用嗎？	常漏掉這件事	要先 opt-in ： 2019 年 3 月 20 日後開通的 Region 預設關閉

中間那欄每個模型都不一樣，這正是重點；**右欄才是答案**。

全課工作流：**AI** 給地圖，文件給答案，你給責任

上色的那一步是唯一能抓錯的地方



課程規矩：程式碼從官方範例來，不從 **AI** 來。交付物裡不能有你解釋不了的行。

每一步這樣問：要位置與理由，不要成品

同一個模型，換個問法，拿到的東西完全不同

✗ 不要這樣問

✓ 這樣問

要地圖

「幫我做 X」

「我要做 X，列出步驟，每步該查的官方文件名稱與官方範例在哪」

驗證

「這樣對嗎？」

「這個參數在文件的哪一頁？給連結」

自己改

「幫我改」

「如果我把 A 改成 B，會發生什麼？」

負責

「幫我解釋一下」

「請當面試官，問我這三行為什麼這樣寫」

什麼可以不驗證，什麼一定要

分界只有一條：錯了要花錢，或跑不起來

✓ 可以放過

- 語法、格式、縮排
- 翻譯
- 命名建議

錯了跑一次就看到

❗ 一定要驗證

- 參數、預設值、限制
- 價格、配額
- **Region**、版本、已改名的 **API**

錯了要花錢，或跑不起來

❓ 灰色地帶

- 架構建議

它給的是平均值，不是你的情境；拿去對

Well-Architected 再決定

LESSON 5

你的 AI 家教

同一個模型，為什麼有人問出來是教科書，有人問出來是家教？

- 一、開場三件事與六種問法
- 二、好的問法怎麼固定下來
- 三、界線：三不做、三失效

開場三件事：少一件，它就用平均值猜你

要具體到有數字、有語言名稱、有期限

LEVEL

我的程度

「寫過兩年 **Java** 後端，沒碰過雲端，會
Linux 基本指令」

少了它會假設你是教科書讀者

GOAL

我的目標

「三週內把 **Flask API** 部署到虛擬機，而
且能講出每一步為什麼」

沒有限期與驗收，它給你大全

KNOWN

我已經知道什麼

「知道虛擬機與容器的差別，不知道私有
網路是什麼」

少了它會從頭講一遍你會的

存成一個檔案就好，每次對話貼上去。第 5 課的作業就是寫你自己的這一份。

六種問法，骨架直接複製

把它從答題機變成陪讀

要地圖

「我要達成 X，列步驟與每步該查的一手文件」

開始之前

要白話

「用我熟的 Y 類比解釋 Z，並說類比在哪裡失效」

看不懂名詞

要比較

「A 與 B 在成本、上手、生命週期三欄比較，附出處」

選型

要反問我

「不要給答案，問我三個問題確認我懂了」

讀完一節

要挑毛病

「當資安稽核員，逐條質疑這份設定」

交付之前

要出處

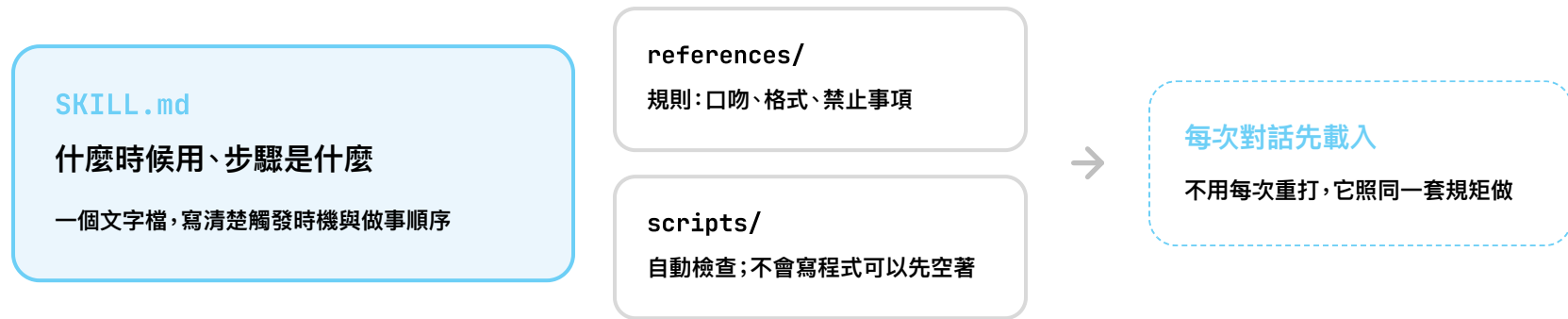
「剛才每個數字的官方頁面在哪，不確定就說不確定」

任何有數字的回答

「要出處」就是第 4 課 workflow 第二步的入口；「要白話」一定要它講出類比的失效點。

好的問法用過三次，就**固定成 Skill**

Skill 是一份給 AI 讀的 SOP，每次對話先載入，它就照同一套規矩做



這幾份教材就是這樣產出來的：先圖後文、名詞先宣告、每課附檢查點，**另一個 AI 再扮小學生挑看不懂的地方。**

界線：整理代勞，思考不代勞

左邊是不要做的三件事，右邊是它會失效的三種情況

× 三件不要做

不要讓它寫交付物

反過來用：讓它改你的草稿、挑你的錯

不要接受看不懂的程式碼

講不出來就刪掉，換個問法重問

不要用它的筆記代替自己寫

反過來用：要它挑你筆記的錯

⚠ 三種情況它會失效

你不知道自己不知道什麼

先讀目錄、考綱、課綱，有地圖才問得出路

需要肌肉記憶

看它做一百次不如自己做十次；第 9 課不准請它代打

需要有人挑戰盲點

它傾向順著你講；要它扮對手，更好的是找真人

QUIZ 2

哪一件事符合「整理代勞，思考不代勞」？

A 請 AI 寫專題的部署腳本直接交

B 請 AI 把你寫的部署腳本逐行挑錯

C 用 AI 生成的章節筆記當自己的筆記

D 看不懂 AI 給的程式碼，先留著之後再說

想 30 秒，等一下翻下一頁對答案。

哪一件事符合「整理代勞，思考不代勞」？

A 請 AI 寫專題的部署腳本直接交

✓ **B** 請 AI 把你寫的部署腳本逐行挑錯

C 用 AI 生成的章節筆記當自己的筆記

D 看不懂 AI 給的程式碼，先留著之後再說

B。你寫，它挑錯；A、C、D 剛好是三件不要做。

Agenda

- 01 判斷資訊：來源、文件、該不該學
- 02 跟 AI 一起學：陷阱與家教
- 03 留得住：記憶與除錯
- 04 做得動：時間與環境

03

第 6、7 課

學了留不住、

卡住出不來，兩件事

LESSON 6

讓東西留在腦子裡

為什麼上課都聽懂了，一週後什麼都不記得？

- 一、 主動回憶與間隔重複
- 二、 這門課怎麼幫你排間隔
- 三、 寫得出來才算懂

重讀最舒服，效用**最低**

高效用的只有兩種：練習測驗與分散練習

✗ 大家通常這樣

重讀筆記、畫重點、影片看第二次

一口氣讀完一章

考前一晚集中複習

✓ 其實應該這樣

闔上書，寫出三個重點，再翻開對

讀一節、闔上、回憶、再下一節

隔天、隔週、隔月各一次，每次都短

主動回憶 (active recall)：不看資料把東西想出來，也就是練習測驗。

間隔重複 (spaced repetition)：隔一段時間再回憶一次，也就是分散練習。

為什麼有效：讀、忘、用力想、再拉長

上色的那一格才是在練；這是我的類比，不是研究原文



讀完一節立刻回憶是檢查有沒有讀進去；隔一段時間再回憶，才是分散練習要的時間。兩個都做。

這門課怎麼幫你排間隔

你不用自己排，只要出現、並且真的闖上教材



寫給不懂的人看，才知道自己哪裡不懂

術語是壓縮檔，寫白話等於解壓縮

- 1 挑一個概念，今天教過的就行
- 2 寫給沒學過的人，不准術語、不准縮寫
- 3 卡住就回去讀，那裡就是你以為會的
- 4 再寫短一半，短得下來才是懂

個人知識庫兩原則，工具不重要

用自己的話寫

每條筆記至少一句是你的判斷；寫不出判斷代表還沒讀懂

留下來源連結

一年後從連結重查；連結失效本身就是第 1 課講的過期訊號

每週回去改舊筆記，**改筆記就是一次主動回憶**。這也是為什麼不要用 AI 生成的筆記代替自己寫：它的筆記是它的理解。

LESSON 7

除錯即學習

卡住的時候，前 30 分鐘該做什麼，30 分鐘後該做什麼？

- 一、 讀錯誤訊息與二分法
- 二、 最小可重現範例三個條件
- 三、 提問四段格式與 **30** 分鐘規則

讀錯誤訊息：最後一行先看

traceback 是程式出錯時走過的呼叫路徑

終端機輸出

```
Traceback (most recent call last):  
  File "/site-packages/lib/run.py", line 88  
    return handler(event)  
  
  File "report.py", line 14, in total  
    amount = amount + row["fee"]  
  
TypeError: unsupported operand type(s)  
for +: 'int' and 'str'
```

- 1 最後一行：看型別與訊息，例如 **TypeError**、**NameError**
- 2 往上找第一個你自己寫的檔案，灰色那幾行是套件裡的，不是你的
- 3 找到那一行，看當時的變數值是什麼

三步做完才去搜尋。搜尋時把你的檔案路徑、變數名拿掉，**只留型別與訊息**。

二分法：每次砍一半

一千行對半砍十次就到一行，前提是每次砍完還是原本那個錯

確認整段會錯

記住最後一行長什麼樣



註解掉後半

還是同一個錯嗎？

還是同一個錯

錯在前半

再砍前半的一半，十行內就會找到那一行

錯誤消失

錯在剛砍掉的後半

放回一半再試

變成別的錯

你切壞了

多半砍掉了被依賴的定義；把那幾行留著
再切

規則只有一條：每一步只改一件事。一次改兩件，好了也不知道是誰的功勞。

最小可重現範例：三個條件

能讓別人重現你的錯的最短程式，也是 **Stack Overflow** 的提問標準

MINIMAL

最小

拿掉所有跟錯誤無關的程式碼，拿掉之後還會錯才算

常見沒做到：整個專案貼上來

COMPLETE

完整

別人貼上就能跑，含 **import**、假資料、版本

常見沒做到：靠「你的環境」才能跑

REPRODUCIBLE

可重現

每次跑都出同一個錯，寫清楚作業系統與版本

常見沒做到：「有時候會錯」

做到第三個條件的時候，**一半的錯你已經自己找到了**：拿掉無關的東西，就是在逼你想這個到底有沒有關係。

提問四段格式：沒寫齊不回答

助教與同學只回應寫齊四段的問題，這是課程規矩

做了什麼

指令或程式碼原文、環境、版本

反例：「我照教學做」

預期什麼

你以為會出現什麼

反例：「應該要成功」

實際什麼

錯誤訊息原文，最後一行必貼

反例：截了一半的畫面

已試過什麼

改過什麼、結果怎樣

反例：「都試過了」

問 AI 也一樣：整個 **traceback** 丟過去說「修好」，改成貼四段，要它先問你三個釐清問題；它改了就跑，改成要它解釋改的那一行為什麼，你再手動改。

30 分鐘規則：到點就停

停損不是放棄，是把問題交出去的正確時間



卡兩小時學到的東西，跟卡 30 分鐘一樣多；多出來的 90 分鐘只是折磨。

QUIZ 3

你把程式後半註解掉，錯誤從 **TypeError** 變成 **NameError**。這代表什麼？

A 錯在前半

B 錯在後半

C 你切壞了，把被依賴的行留著再切

D 程式有兩個錯

想 30 秒，等一下翻下一頁對答案。

你把程式後半註解掉，錯誤從 **TypeError** 變成 **NameError**。這代表什麼？

A 錯在前半

B 錯在後半

✓ 你切壞了，把被依賴的行留著再切

D 程式有兩個錯

C。二分法只認「同一個錯」；變成別的錯，這一刀不算數，要重切。

Agenda

- 01 判斷資訊：來源、文件、該不該學
- 02 跟 AI 一起學：陷阱與家教
- 03 留得住：記憶與除錯
- 04 做得動：時間與環境

04

第 8、9 課

第 9 課是本單元驗收：
只靠官方文件建好環境

LESSON 8

在職學習的時間怎麼分

每週只有幾個小時，課堂、實作、證照怎麼分？落後了先砍哪個？

- 一、 三個桶子：課堂、實作、證照
- 二、 落後時的取捨順序
- 三、 三種可用時數，三種排法

三個桶子：實作要整塊，零碎時間做證照

小時數是建議，指課堂以外、已扣掉兩成緩衝之後的時間

課堂

出席就好，時間固定

上課日

實作

電腦、環境、90 分鐘以上的整塊，每週至少 2 塊

週間晚上前段
週末上午

證照自習

手機就行：考綱、服務對照表、回憶上週三問，每週 3 到 5 小時

通勤、排隊
午休

一週長這樣

	一	二	三	四	五	六	日
通勤							
午休							
整塊							

- 實作，一定整塊 90 分鐘以上
- 零碎時間，只放不需要環境的事
- 落後時第一個砍的那一格

三種可用時數，三種排法

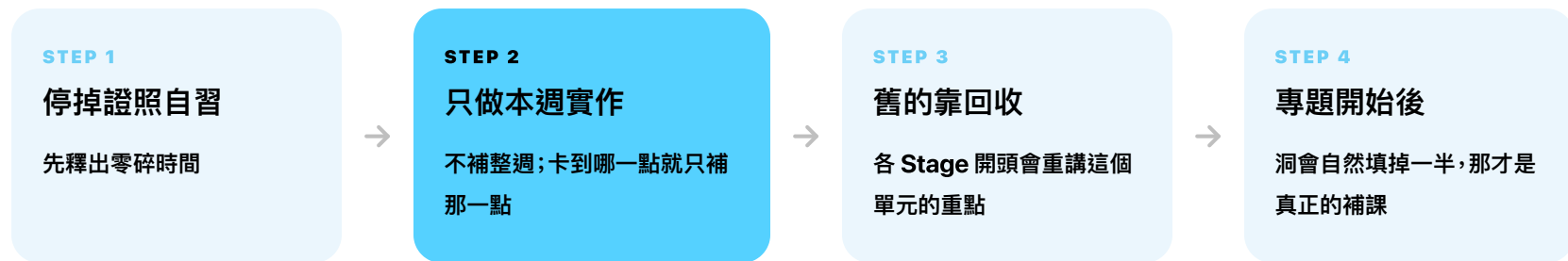
5 小時：只做實作，兩塊各 2.5 小時，證照延後到課程結束

10 小時：實作 2 塊共 6 小時、證照 3 小時零碎、1 小時寫筆記

15 小時以上：實作 3 塊，證照與課程同步考，加一塊做延伸題

落後時的順序：砍證照，保實作，課堂靠回收

專題靠實作，實作不能砍；考試日期是你自己定的



排得完的計畫，才是計畫。先算時數，再減兩成緩衝，剩下的才是預算。

LESSON 9 · 本單元驗收

工作環境建置

50 分鐘內，只靠官方文件，能不能把整門課要用的環境建好？

- 一、 五項驗收標準，各一本文件
- 二、 多組態：一台電腦、多個帳號
- 三、 今天不准搜尋部落格教學

驗收表：每項一條標準、一本文件

MFA 是登入除了密碼還要手機上的一組數字；profile 是 CLI 裡給每組帳號設定取的名字

項目	驗收標準 (做完長這樣)	一手文件 (只給這一本)
雲端帳號	root 已開 MFA；日常不用 root 登入	IAM User Guide：Enable a virtual MFA device for the root user
預算告警	Budgets 列表有 Zero spend 與 Monthly cost 兩筆	Cost Management User Guide：Creating a budget
CLI 與多組態	<code>aws sts get-caller-identity --profile course</code> 回傳你的帳號 ID	CLI User Guide：Configuration and credential file settings；IAM User Guide：Create an IAM user
Python 環境	Python 3.12 以上；venv 或 uv 建好；能 <code>import boto3</code>	venv 文件；uv 文件；Boto3 Quickstart
Git 與版控	<code>git log</code> 有第一個 commit；.gitignore 含 .env 與 *.pem	Pro Git 繁中版第 1、2 章；GitHub Docs：Ignoring files

今天不准搜尋部落格教學，只准一手。卡住可以問 AI 要地圖，但參數回文件對。

金鑰等同你的帳號密碼：不進 repo、不貼給 AI，只貼錯誤訊息。

多組態：一台電腦、多個帳號，不會混用

藍色那段叫 **course**，兩個檔各一段，名字一樣但寫法不同

```
~/aws/config
```

```
[default]
region = ap-northeast-1
output = json
```

```
[profile course]
region = ap-east-2
output = json
```

```
~/aws/credentials
```

```
[default]
aws_access_key_id = ...
aws_secret_access_key = ...
```

```
[course]
aws_access_key_id = ...
aws_secret_access_key = ...
```

常見卡點：看訊號，翻那一頁

command not found: aws → CLI 疑難排解的 **Command not found** 段

which python 指到系統路徑 → **venv** 文件的 **How venvs work**

Unable to locate credentials → 設定檔文件的 **profile** 段

Please tell me who you are → **Pro Git** 初次設定那一節

TAKEAWAY

- 1 在半衰期 12 個月的領域，**會判斷來源**比會背答案值錢；一手下決定，二手找路。
- 2 **想得起來才算會，講得出來才算懂**；筆記自己寫，卡住 30 分鐘就寫四段交出去。
- 3 學什麼是**投資決策**，先看它會不會倒；排得完的計畫，才是計畫。

如果只記得一句話：**AI 給地圖，文件給答案，你給責任。**

Reference

- 1 AWS DevOps Blog ◦ *The Future of AWS CodeCommit* ◦ → [服務生命週期只有官方會提前公告](#)
- 2 AWS Cloud9 User Guide 首頁橫幅；AWS Regions 文件；AWS News Blog 「Now open – AWS Asia Pacific (Taipei) Region」 ◦
→ [第 4 課示範題的正確答案](#)
- 3 AWS Certified Solutions Architect – Associate (SAA-C03) Exam Guide；semver.org 繁中版 ◦ → [考網反推讀書清單、版本號怎麼讀](#)
- 4 Dunlosky, J., Rawson, K. A., Marsh, E. J., Nathan, M. J., & Willingham, D. T. (2013). *Improving Students' Learning with Effective Learning Techniques*. Psychological Science in the Public Interest ◦ → [練習測驗與分散練習為高效用 \(二手引用\)](#)
- 5 Python 官方教學 8.2 Errors and Exceptions；Stack Overflow Help Center 「How to create a Minimal, Reproducible Example」 ◦ → [讀錯誤訊息與最小可重現範例](#)
- 6 AWS CLI User Guide、IAM User Guide、Cost Management User Guide、venv 與 uv 文件、Boto3 Quickstart、Pro Git 繁中版、GitHub Docs ◦ → [第 9 課驗收表指定的那幾本](#)

全部連結 2026-09-12 至 09-13 查閱；課程教材屬二手來源，參數請回一手文件核對。